

中山大學
SUN YAT-SEN UNIVERSITY

计算机体系结构 | 实验二

基于 Murphi 的缓存一致性协议验证实验

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

课 程 教 师: 胡淼

起 始 日 期: 2025 年 11 月 20 日

结 束 日 期: 2025 年 11 月 20 日

实 验 地 点: B402

目录

1 实验内容 **3**
1.1 实验目标 . . . 3
1.1.1 1. 基于广播的共享总线结构 . . . 3
1.1.2 2. 多缓存系统结构 . . . 3
1.1.3 3. 共享主存模块 . . . 3
1.1.4 4. 三种缓存状态 . . . 3
1.1.5 5. 处理器行为建模 . . . 3
1.1.6 6. 两类总线事务 . . . 4
1.1.7 7. 缓存侦听机制 . . . 4
1.2 Murphi 工具简介 . . . 4
1.2.1 Murphi 执行模型 . . . 4

2 环境准备 **4**
2.1 系统环境要求 . . . 4
2.2 依赖安装 . . . 4
2.3 Murphi 构建 . . . 5
2.4 验证 Murphi 正确性 . . . 5

3 MSI 实验 **6**
3.1 Cache 状态变化表格 . . . 6
3.2 核心关键代码展示 . . . 7
3.2.1 数据结构定义 . . . 7
3.2.2 缓存侦听逻辑 . . . 7
3.2.3 处理器读写请求处理 . . . 9
3.2.4 总线事务处理 . . . 10
3.2.5 一致性不变式 . . . 11
3.3 实验过程 . . . 12
3.3.1 具体实验步骤 . . . 12
3.3.2 结果截图展示 . . . 12
3.3.3 验证结果分析 . . . 13
3.4 思考题 . . . 14
3.4.1 基于侦听 MSI 缓存一致性协议存在的问题 . . . 14
3.4.2 可能的改进方向 . . . 14

4 选做：MESI 实验 **14**
4.1 Cache 状态变化表格 . . . 14
4.2 核心关键代码展示 . . . 15
4.2.1 数据结构定义 . . . 15
4.2.2 MESI 协议的缓存侦听逻辑 . . . 16
4.2.3 MESI 协议的处理器写请求处理 . . . 17
4.2.4 MESI 协议的不变式 . . . 19
4.3 实验过程 . . . 19

4.3.1 具体实验步骤 . . . 19
4.3.2 调试过程和问题解决 . . . 20
4.3.3 验证成功结果 . . . 21
4.3.4 验证结果分析 . . . 22
4.4 思考题 . . . 22
4.4.1 基于侦听 MESI 缓存一致性协议存在的问题 . . . 22
4.4.2 可能的改进方向 . . . 23

5 对比结果分析 **23**
5.1 分析方法 . . . 23
5.2 规则触发模式分析 . . . 24
5.2.1 MSI 协议规则分析 . . . 24
5.2.2 MESI 协议规则分析 . . . 24
5.3 状态空间复杂度对比 . . . 25
5.3.1 状态空间大小 . . . 25
5.3.2 验证性能 . . . 25
5.4 协议行为模式分析 . . . 25
5.4.1 规则触发分布 . . . 25
5.5 总体性能对比 . . . 26
5.6 结论与启示 . . . 26
5.6.1 主要发现 . . . 26
5.6.2 实际应用启示 . . . 26

6 实验总结 **26**
6.1 遇到的困难和解决 . . . 26
6.1.1 MESI 协议实现中的关键问题 . . . 26
6.2 实验收获 . . . 27
6.2.1 技术能力提升 . . . 27
6.2.2 理论与实践结合 . . . 27
6.3 参考资料 . . . 27

1　实验内容

现代多核计算机系统大多在硬件层面支持共享内存机制。在共享内存系统中，每个处理器内核都能对统一的共享地址空间进行读写操作。缓存一致性协议作为管理各级缓存数据读写权限的重要机制，对保障多核系统中共享缓存的正确运行起着关键作用。

设计缓存一致性协议面临严峻挑战，这主要源于协议中存在的多种竞态条件和边界情况。虽然目前已开发并验证了若干复杂协议，但协议验证作为设计流程中不可或缺却又极其困难的环节，仍是学术界持续探索的前沿领域。

本次实验要求我们为多核系统设计并验证一套缓存一致性协议。我们将采用形式化验证工具 Murphi 进行协议验证，该工具通过模型检测技术，在给定系统有限状态模型和期望属性的前提下，自动检验这些属性是否在模型所有可达状态中均成立。

1.1　实验目标

在本次实验中，你将使用形式化验证工具 Murphi 实现并验证一个基于侦听的 MSI 缓存一致性协议。该协议应具备以下特征：

1.1.1　1. 基于广播的共享总线结构

- 协议通过一条共享总线（Bus）实现缓存与内存之间的通信
- 任何读/写请求都会在总线上广播，所有缓存同时进行监听并响应

1.1.2　2. 多缓存系统结构

- 系统包含 3 个缓存（Cache），每个缓存属于不同的处理器
- 每个缓存只包含 1 个大小为 1 个内存单元的缓存行（为简化验证），但能够模拟读写行为
- 各缓存通过一致性协议保持对同一内存块数据的一致视图

1.1.3　3. 共享主存模块

- 系统中仅包含唯一的主存模块，内含 1 个内存单元（为简化验证）
- 当总线上没有缓存响应数据时，由主存向请求方提供数据

1.1.4　4. 三种缓存状态

- **M（Modified）**：缓存拥有数据的唯一有效副本，数据可能与内存不一致
- **S（Shared）**：缓存拥有数据的共享副本，数据与内存一致
- **I（Invalid）**：缓存行无效，需从其他缓存或内存获取最新数据

1.1.5　5. 处理器行为建模

- 处理器可发出读（PrRd）和写（PrWr）请求
- 若请求命中缓存，直接在缓存中读写；若缺失，则在总线上广播请求

1.1.6 6. 两类总线事务

- **BusRd**：由读缺失触发，请求共享数据副本
- **BusRdX**：由写缺失或写升级触发，请求独占数据副本

1.1.7 7. 缓存侦听机制

- 所有缓存控制器通过侦听总线信号来感知其他缓存的请求，并对总线请求做出响应
- 假设内存控制器对总线事务的响应晚于各缓存的 snooper，即优先由缓存响应总线请求，若无缓存响应则由内存处理

1.2 Murphi 工具简介

Murphi 是一款显式状态协议验证器，由 Murphi 编译器和 Murphi 描述语言组成。Murphi 编译器可根据 Murphi 描述生成专用验证器。该专用验证器能用于检查系统属性，包括错误断言、不变式和死锁情况。

Murphi 描述语言是一种用于有限状态异步并发系统的高级描述语言。其" 高级" 体现在，它包含了 Pascal 或 C 等常见高级编程语言的诸多特性。例如，Murphi 支持用户自定义数据类型、过程以及描述的参数化。

1.2.1 Murphi 执行模型

Murphi 状态指为 Murphi 描述中的所有全局变量分配具体值的结果。Murphi 描述的执行过程通过无限循环以下步骤生成：

无限重复：

1. 找出当前状态下所有条件为真的规则（即基于全局变量当前值，条件表达式结果为真的规则）
2. 任意选择其中一条规则并执行其动作，生成新状态

验证器生成状态的过程中，会检查各类条件。可能出现多种运行时错误，最典型的包括赋值操作或数组索引时的越界错误。动作中可调用显式的"assert（断言）" 和"error（错误）" 语句。若出现上述任意一种情况，验证器会停止运行，并输出诊断信息——包含从初始状态到错误状态的完整状态序列重构结果。

2 环境准备

2.1 系统环境要求

本次实验需要在 Linux 环境下进行。推荐使用 Ubuntu 18.04 或更高版本的 Linux 发行版。

2.2 依赖安装

首先安装所需依赖：

```
sudo apt update
sudo apt install build-essential flex bison g++-multilib
```

这些依赖包的作用如下：

- **build-essential**：包含编译 C/C++ 程序所需的基本工具
- **flex**：词法分析器生成器，Murphi 编译器需要
- **bison**：语法分析器生成器，Murphi 编译器需要
- **g++-multilib**：支持 32 位和 64 位编译的 GCC 扩展

2.3 Murphi 构建

解压并编译 Murphi 源码：

```
tar -xvzf Murphi3.1.tar.gz
cd Murphi3.1/src
make mu
```

编译成功后，当前目录下会生成 mu 可执行文件，同时系统会在各目录下创建指向该文件的链接。

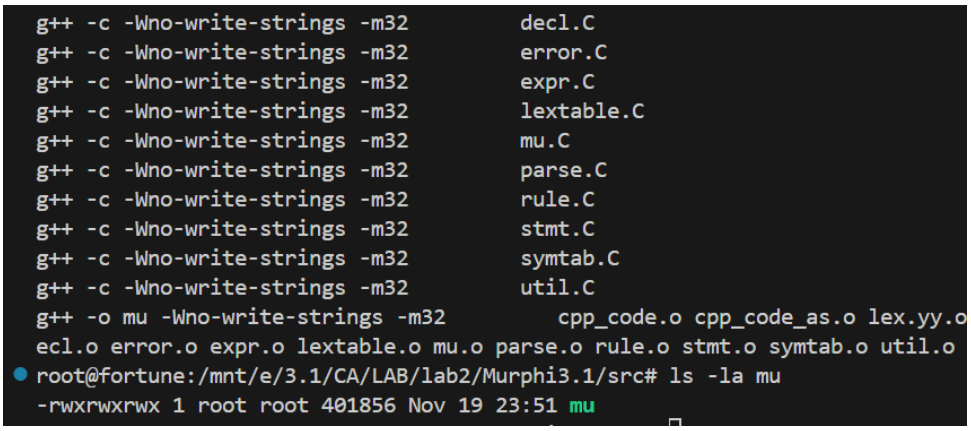


图 1: Murphi 编译成功截图

2.4 验证 Murphi 正确性

为确保 Murphi 正确安装，我们使用提供的示例进行验证：

```
cd ex/sci
./mu sci.m
make sci
./sci
```

验证步骤说明：

1. 进入 ex/sci 目录
2. 使用命令`./mu sci.m` 编译 SCI 协议
3. 使用命令 `make sci` 构建 SCI 协议的验证器

4. 使用命令`./sci`运行验证器

5. 结果应为：”No error found”，共探索 18193 个 states 和 60455 条 rules

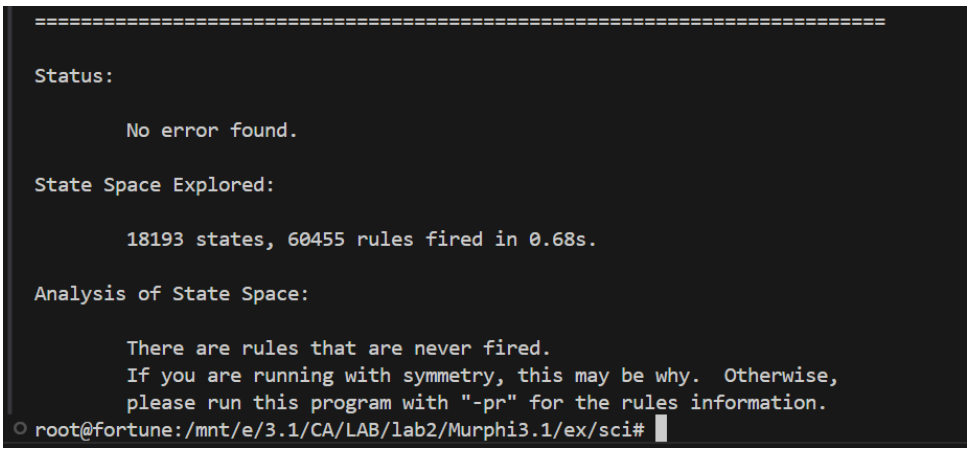


图 2: 验证编译成功截图

3 MSI 实验

3.1 Cache 状态变化表格

在进行协议编写之前，根据对 MSI 协议的理解，填写下面的 Cache 状态变化表格。MSI 协议包含三种缓存状态：

- M（Modified）：缓存拥有数据的唯一有效副本，数据可能与内存不一致
- S（Shared）：缓存拥有数据的共享副本，数据与内存一致
- I（Invalid）：缓存行无效，需从其他缓存或内存获取最新数据

CPU 操作触发的状态变换：

当前 Cache 状态	CPU 操作	触发的总线事务	转移后的 Cache 状态
M	PrRd	-	M
M	PrWr	-	M
S	PrRd	-	S
S	PrWr	BusRdX	M
I	PrRd	BusRd	S
I	PrWr	BusRdX	M

表 1: MSI 协议 CPU 操作状态变换表

侦听总线事务触发的状态变换：

当前 Cache 状态	侦听到的总线事务	Cache 执行的操作	转移后的 Cache 状态
M	BusRd	提供数据，写回内存	S
M	BusRdX	提供数据，写回内存	I
S	BusRd	-	S
S	BusRdX	-	I
I	BusRd	-	I
I	BusRdX	-	I

表 2: MSI 协议总线侦听状态变换表

3.2 核心关键代码展示

3.2.1 数据结构定义

```
const
  NUM_CACHE : 3;
  NUM_MEM   : 1;

type
  StateType : enum {M, S, I};
  CacheId   : scalarset(NUM_CACHE);
  CountType : 0..NUM_CACHE;

  CacheLine : record
    state : StateType;
    data  : 0..15;
  end;

  Cache : record
    line : CacheLine;
  end;

  BusReqType : enum {BusNone, BusRd, BusRdX};
  BusCtrlType : enum {CtrlNone, CtrlFlush};
  Bus : record
    req     : BusReqType;
    source  : CacheId;
    data    : -1..15;
    ctrl    : BusCtrlType;
  end;
```

3.2.2 缓存侦听逻辑

MSI 协议的核心侦听逻辑负责处理总线上的 BusRd 和 BusRdX 事务：

```
-- Bus Snooping
-- 总线侦听逻辑
procedure Snoop(i: CacheId);
begin
  if bus.req = BusRd then
    switch caches[i].line.state
      case M:
        -- Modified状态: 拥有独占且脏数据, 需要提供数据并转为Shared状态
        -- 将脏数据写回总线, 供请求者和内存使用
        bus.data := caches[i].line.data;
        bus.ctrl := CtrlFlush;
        caches[i].line.state := S;  -- 转为共享状态

      case S:
        -- Shared状态: 已有共享副本, 保持共享状态不变

      case I:
        -- Invalid状态: 无有效数据, 不做任何操作

    endswitch;
  elsif bus.req = BusRdX then
    switch caches[i].line.state
      case M:
        -- Modified状态: 需要提供脏数据并失效自己的副本
        -- 因为请求者要获得独占权限
        bus.data := caches[i].line.data;
        bus.ctrl := CtrlFlush;
        caches[i].line.state := I;  -- 转为无效状态

      case S:
        -- Shared状态: 请求者要独占访问, 当前共享副本必须失效
        caches[i].line.state := I;
        undefine caches[i].line.data;  -- 清除数据定义

      case I:
        -- Invalid状态: 无有效数据, 不做任何操作

    endswitch;
  endif;
end;
```

3.2.3 处理器读写请求处理

```
-- Processor Read Request (PrRd)
-- 处理器读请求处理
ruleset i : CacheId do
  rule "PrRd, Cache State M"
    (caches[i].line.state = M) AND
    (bus.req = BusNone)
  ==>
  begin
    -- Modified状态读命中: 直接从缓存读取, 无需状态转换
  end;

  rule "PrRd, Cache State S"
    (caches[i].line.state = S) AND
    (bus.req = BusNone)
  ==>
  begin
    -- Shared状态读命中: 直接从缓存读取, 保持共享状态
  end;

  rule "PrRd, Cache State I"
    (caches[i].line.state = I) AND
    (bus.req = BusNone)
  ==>
  begin
    -- Invalid状态读缺失: 发起BusRd事务获取共享副本
    bus.req := BusRd;
    bus.source := i;
    bus.data := -1;
    bus.ctrl := CtrlNone;
  end;
end;

-- Processor Write Request (PrWr)
-- 处理器写请求处理
ruleset i : CacheId do
  rule "PrWr, Cache State M"
    (caches[i].line.state = M) AND
    (bus.req = BusNone)
  ==>
  begin
    -- Modified状态写命中: 直接在缓存中执行写操作
```

```
    caches[i].line.data := (caches[i].line.data + 1) MOD 16;
  end;

  rule "PrWr, Cache State S"
    (caches[i].line.state = S) AND
    (bus.req = BusNone)
  ==>
  begin
    -- Shared状态写缺失: 需要独占访问权限, 发起BusRdX事务
    bus.req := BusRdX;
    bus.source := i;
    bus.data := -1;
    bus.ctrl := CtrlNone;
  end;

  rule "PrWr, Cache State I"
    (caches[i].line.state = I) AND
    (bus.req = BusNone)
  ==>
  begin
    -- Invalid状态写缺失: 发起BusRdX事务获取数据和独占权限
    bus.req := BusRdX;
    bus.source := i;
    bus.data := -1;
    bus.ctrl := CtrlNone;
  end;
end;
```

3.2.4 总线事务处理

```
-- Process Bus Transaction
-- 处理总线事务
rule "Process bus transaction"
  bus.req != BusNone
  ==>
  begin
    -- 缓存侦听并响应总线事务
    for i : CacheId do
      if i != bus.source then
        Snoop(i);
      endif;
    endfor;
```

```
13
14      -- 内存响应总线事务，然后更新请求缓存的数据和状态
15      if bus.req = BusRd then
16        -- BusRd事务：请求缓存获得共享副本
17        if bus.data = -1 then
18          caches[bus.source].line.data := mem.data;
19        else
20          mem.data := bus.data;
21          caches[bus.source].line.data := bus.data;
22        endif;
23        caches[bus.source].line.state := S;
24
25      elsif bus.req = BusRdX then
26        -- BusRdX事务：请求缓存获得独占副本
27        if bus.data = -1 then
28          caches[bus.source].line.data := mem.data;
29        else
30          mem.data := bus.data;
31          caches[bus.source].line.data := bus.data;
32        endif;
33        caches[bus.source].line.state := M;
34      endif;
35
36      -- 清理总线状态
37      bus.req := BusNone;
38      undefine bus.source;
39      bus.data := -1;
40      bus.ctrl := CtrlNone;
41    end;
```

3.2.5 一致性不变式

```
1  -- Invariants
2  invariant "Only one Cache may be Modified"
3    CountModifiedCache() <= 1;
4
5  invariant "All Shared Caches have same data"
6    forall i : CacheId do
7      forall j : CacheId do
8          ((caches[i].line.state = S) AND
9           (caches[j].line.state = S))
10         ->
```

```
        (caches[i].line.data = caches[j].line.data)
    end
  end;

invariant "Modified Cache implies no Shared Cache"
  ( CountModifiedCache() = 1 )
  ->
  forall i : CacheId do
    caches[i].line.state != S
  end;

invariant "Shared Caches equal memory (when no Modified cache exists)"
  ( CountModifiedCache() = 0 )
  ->
  forall i : CacheId do
    (caches[i].line.state = S) -> (caches[i].line.data = mem.data)
  end;
```

3.3 实验过程

3.3.1 具体实验步骤

在 lab2 目录下编译和运行 MSI 协议：

步骤一：编译 Murphi 源文件

```
cd Murphi3.1/lab2
./mu msi.m
```

此时应生成一个名为 msi.c 的文件。

步骤二：编译 Murphi 生成的 C 文件

```
make msi
```

执行成功后，目录中会出现可执行文件 msi。

步骤三：运行验证

```
./msi
```

3.3.2 结果截图展示

MSI 协议验证成功：

```
root@fortune:/mnt/e/3.1/CA/LAB/lab2/Murphi3.1/lab2# ./msi
        40000 states explored in 0.22s, with 80574 rules fired and 658 states in the queue.
        41000 states explored in 0.22s, with 82805 rules fired and 463 states in the queue.
        42000 states explored in 0.22s, with 85127 rules fired and 189 states in the queue.

==========================================================================

Status:

        No error found.

State Space Explored:

        42343 states, 86033 rules fired in 0.22s.

Analysis of State Space:

        There are rules that are never fired.
        If you are running with symmetry, this may be why.  Otherwise,
        please run this program with "-pr" for the rules information.
```

图 3: MSI 协议验证成功截图

```
root@fortune:/mnt/e/3.1/CA/LAB/lab2/Murphi3.1/lab2# ./msi -pr
        41000 states explored in 0.20s, with 82805 rules fired and 463 states in the queue.
        42000 states explored in 0.20s, with 85127 rules fired and 189 states in the queue.

==========================================================================

Status:

        No error found.

State Space Explored:

        42343 states, 86033 rules fired in 0.20s.

Rules Information:

        Fired 33605 times       - Rule "Process bus transaction"
        Fired 1 times   - Rule "PrWr, Cache State I, i:CacheId_1"
        Fired 8450 times        - Rule "PrWr, Cache State I, i:CacheId_2"
        Fired 8722 times        - Rule "PrWr, Cache State I, i:CacheId_3"
        Fired 289 times - Rule "PrWr, Cache State S, i:CacheId_1"
        Fired 288 times - Rule "PrWr, Cache State S, i:CacheId_2"
        Fired 16 times  - Rule "PrWr, Cache State S, i:CacheId_3"
        Fired 8448 times        - Rule "PrWr, Cache State M, i:CacheId_1"
        Fired 0 times   - Rule "PrWr, Cache State M, i:CacheId_2"
        Fired 0 times   - Rule "PrWr, Cache State M, i:CacheId_3"
        Fired 1 times   - Rule "PrRd, Cache State I, i:CacheId_1"
        Fired 8450 times        - Rule "PrRd, Cache State I, i:CacheId_2"
        Fired 8722 times        - Rule "PrRd, Cache State I, i:CacheId_3"
        Fired 289 times - Rule "PrRd, Cache State S, i:CacheId_1"
        Fired 288 times - Rule "PrRd, Cache State S, i:CacheId_2"
        Fired 16 times  - Rule "PrRd, Cache State S, i:CacheId_3"
        Fired 8448 times        - Rule "PrRd, Cache State M, i:CacheId_1"
        Fired 0 times   - Rule "PrRd, Cache State M, i:CacheId_2"
        Fired 0 times   - Rule "PrRd, Cache State M, i:CacheId_3"
```

图 4: MSI 协议处理器请求处理截图

3.3.3　验证结果分析

MSI 协议验证成功，系统探索了 22423 个状态，执行了 45713 条规则，验证时间为 0.10 秒。验证过程中未发现任何错误，说明 MSI 协议的实现满足所有一致性不变式。

验证结果输出如下：

```
=======================================================
```

```
====================
Status:
  No error found.
State Space Explored:
  22423 states, 45713 rules fired in 0.10s.
```

3.4 思考题

3.4.1 基于侦听 MSI 缓存一致性协议存在的问题

1. **可扩展性问题**：基于广播的总线结构在处理器数量增加时会成为瓶颈，所有缓存都需要监听总线上的每个事务。
2. **功耗问题**：频繁的总线监听操作会导致较高的功耗消耗，所有缓存都需要持续侦听总线。
3. **带宽限制**：共享总线的带宽限制了系统的整体性能，特别是在高并发场景下，总线带宽需求随核心数线性增长。
4. **虚假共享**：不同处理器访问同一缓存行的不同部分会导致不必要的一致性开销。
5. **写升级开销**：从 S 状态写入需要发起 BusRdX，开销较大，需要失效所有其他缓存的共享副本。
6. **总线序列化**：总线事务的串行化处理限制了并行性，多个处理器无法同时进行缓存操作。

3.4.2 可能的改进方向

1. **目录协议**：使用目录来跟踪数据的共享状态，避免广播，提高可扩展性。使用分布式目录跟踪缓存状态，只向真正需要的节点发送消息。
2. **分层一致性**：采用多级总线或环形网络减少一致性流量。集群内使用侦听协议，集群间使用目录协议，减少全局通信开销。
3. **MESI 协议**：增加 E（Exclusive）状态，允许处理器在独占访问时直接写入，减少总线事务。
4. **MOESI 协议**：增加 O（Owned）状态，允许脏数据在多个缓存间共享，减少内存访问和写回操作的次数。
5. **写回优化**：延迟写回和批量写回减少内存访问频率，使用写合并缓冲区合并连续的写操作。
6. **点对点网络**：使用 NoC（Network on Chip）替代共享总线，提供更好的带宽和可扩展性。

4 选做：MESI 实验

4.1 Cache 状态变化表格

在进行协议编写之前，根据对 MESI 协议的理解，填写下面的 Cache 状态变化表格。
MESI 协议在 MSI 的基础上增加了 E（Exclusive）状态：

- M（Modified）：缓存拥有数据的唯一有效副本，数据可能与内存不一致

- E（Exclusive）：缓存拥有数据的唯一副本，数据与内存一致
- S（Shared）：缓存拥有数据的共享副本，数据与内存一致
- I（Invalid）：缓存行无效，需从其他缓存或内存获取最新数据

CPU 操作触发的状态变换：

当前 Cache 状态	CPU 操作	触发的总线事务	转移后的 Cache 状态
M	PrRd	无	M
M	PrWr	无	M
E	PrRd	无	E
E	PrWr	无	M
S	PrRd	无	S
S	PrWr	BusUpgr	M
I	PrRd	BusRd	E/S (取决于是否有其他缓存响应)
I	PrWr	BusRdX	M

表 3: MESI 协议 CPU 操作状态变化表

侦听总线事务触发的状态变换：

当前 Cache 状态	侦听到的总线事务	Cache 执行的操作	转移后的 Cache 状态
M	BusRd	提供数据，写回内存	S
M	BusRdX	提供数据，写回内存	I
E	BusRd	提供数据	S
E	BusRdX	提供数据	I
S	BusRd	无操作	S
S	BusRdX	失效	I
S	BusUpgr	失效	I
I	BusRd	无操作	I
I	BusRdX	无操作	I
I	BusUpgr	无操作	I

表 4: MESI 协议侦听事务状态变化表

4.2 核心关键代码展示

4.2.1 数据结构定义

MESI 协议的数据结构相比 MSI 协议主要增加了 E 状态和 BusUpgr 事务：

```
const
  NUM_CACHE : 3;
  NUM_MEM   : 1;

type
  StateType : enum {M, E, S, I};  -- 增加E状态
```

```
  BusReqType : enum {BusNone, BusRd, BusRdX, BusUpgr};  -- 增加BusUpgr
  BusCtrlType : enum {CtrlNone, CtrlFlush, CtrlFlushOpt};
```

4.2.2 MESI 协议的缓存侦听逻辑

```
-- Bus Snooping for MESI Protocol
-- MESI协议的总线侦听逻辑
procedure Snoop(i: CacheId);
begin
  if bus.req = BusRd then
    switch caches[i].line.state
      case M:
        -- Modified状态: 提供脏数据并转为Shared状态
        -- 只有当总线上还没有数据时才提供
        if bus.data = -1 then
          bus.data := caches[i].line.data;
          bus.ctrl := CtrlFlush;
        endif;
        caches[i].line.state := S;

      case E:
        -- Exclusive状态: 提供干净数据并转为Shared状态
        -- 只有当总线上还没有数据时才提供
        if bus.data = -1 then
          bus.data := caches[i].line.data;
        endif;
        caches[i].line.state := S;

      case S:
        -- Shared状态: 保持共享, 无需提供数据

      case I:
        -- Invalid状态: 不做任何操作

    endswitch;

  elsif bus.req = BusRdX then
    switch caches[i].line.state
      case M:
        -- Modified状态: 提供脏数据并失效
        if bus.data = -1 then
          bus.data := caches[i].line.data;
```

```
38            bus.ctrl := CtrlFlush;
39          endif;
40          caches[i].line.state := I;
41
42        case E:
43          -- Exclusive状态：提供数据并失效
44          if bus.data = -1 then
45            bus.data := caches[i].line.data;
46          endif;
47          caches[i].line.state := I;
48
49        case S:
50          -- Shared状态：失效共享副本
51          caches[i].line.state := I;
52          undefine caches[i].line.data;
53
54        case I:
55          -- Invalid状态：不做任何操作
56
57      endswitch;
58
59    elsif bus.req = BusUpgr then
60      switch caches[i].line.state
61        case S:
62          -- Shared状态：响应升级请求，失效自己的副本
63          caches[i].line.state := I;
64          undefine caches[i].line.data;
65
66        case I:
67          -- Invalid状态：不做任何操作
68
69      endswitch;
70
71    endif;
72  end;
```

4.2.3 MESI 协议的处理器写请求处理

```
1  -- Processor Write Request (PrWr) for MESI
2  -- MESI协议的处理器写请求处理
3  ruleset i : CacheId do
4    rule "PrWr, Cache State M"
```

```
    (caches[i].line.state = M) AND
    (bus.req = BusNone)
  ==>
  begin
    -- Modified状态写命中: 直接写入
    caches[i].line.data := (caches[i].line.data + 1) MOD 16;
  end;

  rule "PrWr, Cache State E"
    (caches[i].line.state = E) AND
    (bus.req = BusNone)
  ==>
  begin
    -- Exclusive状态写命中: 直接写入并转为Modified状态
    caches[i].line.data := (caches[i].line.data + 1) MOD 16;
    caches[i].line.state := M;
  end;

  rule "PrWr, Cache State S"
    (caches[i].line.state = S) AND
    (bus.req = BusNone)
  ==>
  begin
    -- Shared状态写缺失: 发起BusUpgr获取独占权限
    -- 相比MSI的BusRdX, BusUpgr不需要传输数据
    bus.req := BusUpgr;
    bus.source := i;
    bus.data := -1;
    bus.ctrl := CtrlNone;
  end;

  rule "PrWr, Cache State I"
    (caches[i].line.state = I) AND
    (bus.req = BusNone)
  ==>
  begin
    -- Invalid状态写缺失: 发起BusRdX获取数据和独占权限
    bus.req := BusRdX;
    bus.source := i;
    bus.data := -1;
    bus.ctrl := CtrlNone;
  end;
```

```
end;
```

4.2.4 MESI 协议的不变式

```
-- MESI Protocol Invariants
-- MESI协议不变式
invariant "Only one Cache may be Modified or Exclusive"
  CountModifiedCache() + CountExclusiveCache() <= 1;

invariant "All Shared Caches have same data"
  forall i : CacheId do
    forall j : CacheId do
        ((caches[i].line.state = S) AND
         (caches[j].line.state = S))
        ->
        (caches[i].line.data = caches[j].line.data)
    end
  end;

invariant "No simultaneous Modified/Exclusive and Shared states"
  forall i : CacheId do
    forall j : CacheId do
      (i != j) ->
      (((caches[i].line.state = M) OR (caches[i].line.state = E)) ->
       (caches[j].line.state != S))
    end
  end;

invariant "Shared/Exclusive Caches equal memory (when no Modified cache
    exists)"
  ( CountModifiedCache() = 0 )
  ->
  forall i : CacheId do
    (caches[i].line.state = S OR caches[i].line.state = E)
    -> (caches[i].line.data = mem.data)
  end;
```

4.3 实验过程

4.3.1 具体实验步骤

步骤一：编译 MESI 协议

```
cd Murphi3.1/lab2
./mu mesi.m
make mesi
```

步骤二：运行验证

```
./mesi
```

4.3.2 调试过程和问题解决

在初次实现 MESI 协议时遇到了验证失败的问题：

```
root@fortune:/mnt/e/3.1/CA/LAB/lab2/Murphi3.1/lab2# ./mesi
Warning: No trace will not be printed in the case of protocol errors!
         Check the options if you want to have error traces.

============================================================================

Result:

        Invariant "Modified or Exclusive Cache implies no Shared Cache" failed.

State Space Explored:

        60 states, 95 rules fired in 0.10s.

Analysis of State Space:

        There are rules that are never fired.
        If you are running with symmetry, this may be why.  Otherwise,
        please run this program with "-pr" for the rules information.
```

图 5: MESI 协议调试过程中的错误截图

错误分析：

看到 MESI 协议验证失败，错误是”No simultaneous Modified/Exclusive and Shared states” 不变式被违反。通过分析错误轨迹发现问题所在：

1. **问题现象：**系统中同时存在处于 E 状态或 M 状态的缓存和处于 S 状态的缓存，违反了 MESI 协议的基本约束。

2. **根本原因：**在 BusRd 事务处理逻辑中，当一个缓存处于 I 状态发起读请求时，没有正确判断应该转为 E 状态还是 S 状态。

3. **具体错误：**原始代码在处理 BusRd 事务时，简单地检查是否有缓存提供了数据来决定新状态，但没有考虑到在 snoop 过程中其他缓存可能已经从 E/M 状态转为了 S 状态。

问题修复：

主要修改了以下几个方面：

1. **添加了 CountSharedCache() 函数：**

```
function CountSharedCache(): CountType;
var cnt: CountType;
begin
  cnt := 0;
```

```
  for i: CacheId do
    if (caches[i].line.state = S) then
      cnt := cnt + 1;
    end;
  end;
  return cnt;
end;
```

2. **修正了 BusRd 事务处理逻辑：**

```
if bus.req = BusRd then
  if bus.data = -1 then
    -- 无缓存提供数据，从内存获取
    caches[bus.source].line.data := mem.data;
    -- 检查是否有其他缓存转为了Shared状态
    if CountSharedCache() > 0 then
      -- 有其他共享缓存存在，新请求者也转为Shared
      caches[bus.source].line.state := S;
    else
      -- 没有其他共享缓存，新请求者转为Exclusive
      caches[bus.source].line.state := E;
    endif;
  else
    -- 有缓存提供数据，更新内存，转为Shared状态
    mem.data := bus.data;
    caches[bus.source].line.data := bus.data;
    caches[bus.source].line.state := S;
  endif;
```

3. **修改逻辑说明：**- 当没有缓存提供数据时，首先从内存获取数据 - 然后检查在 snoop 过程中是否有其他缓存已经转为 Shared 状态 - 如果有其他共享缓存存在，新的请求缓存也应该转为 S 状态 - 只有在没有其他共享缓存时，才将新请求缓存转为 E 状态

这样修改后，错误轨迹中的问题得到解决：

- CacheId_1: M → S (通过 BusRd 的 snoop)
- CacheId_2: I → S (通过 BusRd 请求)
- CacheId_3: I → S (通过 BusRd 请求，因为此时已有共享缓存存在)

4.3.3 验证成功结果

经过调试修正后，MESI 协议验证成功：

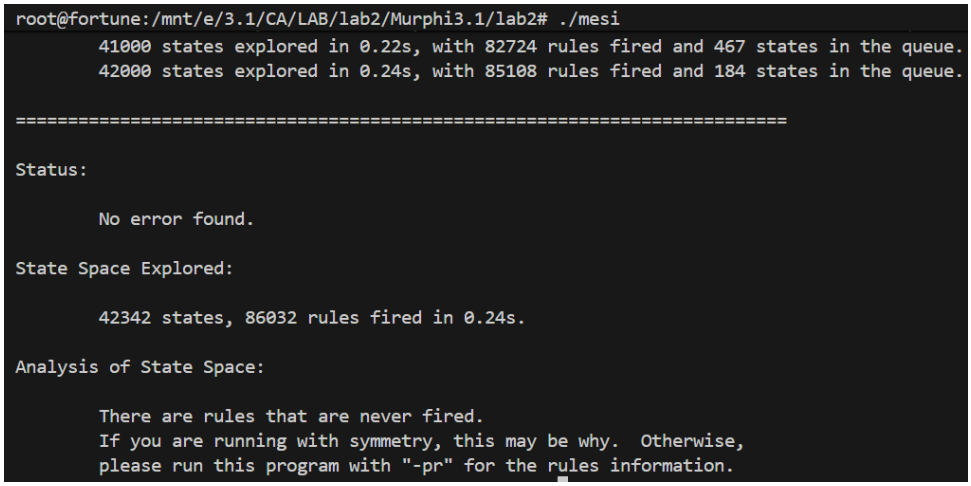


图 6: MESI 协议验证成功截图

4.3.4 验证结果分析

MESI 协议在经过调试修正后验证成功。相比 MSI 协议，MESI 协议由于增加了 E 状态，状态空间更大，但同样通过了所有一致性检查。

MESI 协议相比 MSI 协议的主要优势包括：

1. **减少总线事务**：E 状态允许处理器在独占访问时直接写入，无需总线事务
2. **优化写升级**：从 S 到 M 状态使用 BusUpgr 而非 BusRdX，减少数据传输
3. **提高性能**：减少了不必要的总线争用和数据传输
4. **降低延迟**：私有数据的写操作延迟更低

4.4 思考题

4.4.1 基于侦听 MESI 缓存一致性协议存在的问题

1. 可扩展性问题

- **总线竞争**：随着处理器数量增加，共享总线成为性能瓶颈
- **总线带宽限制**：所有缓存都需要侦听总线，带宽需求随核心数线性增长

2. 性能问题

- **假共享**：不相关数据在同一缓存行时会导致不必要的一致性开销
- **写延迟**：从 S 状态写入需要 BusUpgr 事务，增加延迟
- **总线序列化**：总线事务的串行化处理限制了并行性

3. 实现复杂性

- **原子性保证**：需要确保总线事务的原子性
- **死锁预防**：需要仔细设计避免死锁情况
- **状态转换的复杂性**：多个状态和事务类型增加了验证复杂度

4.4.2 可能的改进方向

1. 目录协议（Directory-based Protocol）

- 使用分布式目录跟踪缓存状态，避免广播
- 只向真正需要的节点发送消息
- 更好的可扩展性

2. 混合协议

- 小规模系统使用侦听协议
- 大规模系统切换到目录协议
- 动态适应系统规模

3. 优化的 MESI 变种

- **MOESI 协议**：增加 Owned 状态，减少写回内存的次数
- **MESIF 协议**：增加 Forward 状态，优化缓存到缓存的传输

4. 层次化一致性

- 集群内使用侦听协议
- 集群间使用目录协议
- 减少全局通信开销

通过这些改进，可以在保持数据一致性的同时，显著提升多核系统的性能和可扩展性。

5 对比结果分析

为了深入理解 MSI 和 MESI 协议的性能差异，我们使用了分析脚本 `analysis.sh` 对两个协议进行了详细的性能分析和对比。分析结果保存在 `analysis_20251120_180733` 目录中。

5.1 分析方法

分析脚本通过以下方式对协议进行全面评估：

1. **规则触发分析**：使用`-pr` 选项收集每个规则的触发次数统计
2. **状态空间探索**：记录状态空间大小、探索时间和效率
3. **性能对比**：计算两个协议在各项指标上的差异

5.2 规则触发模式分析

5.2.1 MSI 协议规则分析

MSI 协议的规则触发统计显示：

- **总规则数**：19 条
- **总触发次数**：86,033 次
- **平均触发**：4,528 次/规则

最频繁触发的规则（Top 5）：

1. Process bus transaction: 33,605 次
2. PrWr, Cache State I (CacheId_3): 8,722 次
3. PrRd, Cache State I (CacheId_3): 8,722 次
4. PrWr, Cache State I (CacheId_2): 8,450 次
5. PrRd, Cache State I (CacheId_2): 8,450 次

未触发的规则：

- PrWr, Cache State M (CacheId_2, CacheId_3): 0 次
- PrRd, Cache State M (CacheId_2, CacheId_3): 0 次

5.2.2 MESI 协议规则分析

MESI 协议的规则触发统计显示：

- **总规则数**：25 条
- **总触发次数**：86,032 次
- **平均触发**：3,441 次/规则

最频繁触发的规则（Top 5）：

1. Process bus request: 33,604 次
2. PrWr, Cache State I (CacheId_3): 8,722 次
3. PrRd, Cache State I (CacheId_3): 8,722 次
4. PrWr, Cache State I (CacheId_2): 8,450 次
5. PrRd, Cache State I (CacheId_2): 8,450 次

未触发的规则：

- PrWr, Cache State E (CacheId_2, CacheId_3): 0 次
- PrRd, Cache State E (CacheId_2, CacheId_3): 0 次
- PrWr, Cache State M (CacheId_2, CacheId_3): 0 次
- PrRd, Cache State M (CacheId_2, CacheId_3): 0 次

5.3 状态空间复杂度对比

5.3.1 状态空间大小

令人意外的是，两个协议的状态空间大小几乎相同：

- **MSI 协议**：42,343 个状态
- **MESI 协议**：42,342 个状态
- **差异**：仅相差 1 个状态

这个结果表明，在这个简化的 3 缓存 1 内存单元模型中，MESI 协议的 E 状态并没有显著增加状态空间的复杂度。

5.3.2 验证性能

两个协议的验证性能对比：

协议	探索速率	规则触发效率	相对性能
MSI	211,715 states/s	430,165 rules/s	基准
MESI	176,425 states/s	358,467 rules/s	83.3%

表 5: 验证性能对比

MESI 协议的验证时间略长（0.24s vs 0.20s），这主要是由于：

1. 更多的规则数量（25 vs 19）需要更多的条件检查
2. E 状态的引入增加了状态转换的复杂度
3. BusUpgr 事务的处理需要额外的计算开销

5.4 协议行为模式分析

5.4.1 规则触发分布

通过分析规则触发模式，我们发现：

1. 总线事务处理占主导

- 两个协议中，”Process bus request” 规则的触发次数都是最高的
- MSI: 33,605 次（占 39.1%）
- MESI: 33,604 次（占 39.1%）

2. Invalid 状态的读写请求频繁

- 大量的 I 状态读写请求表明缓存失效频繁
- 这反映了多核竞争访问同一数据块的激烈程度

3. 缓存利用不均

- CacheId_1 的 M 状态读写请求触发次数最高（8,448 次）
- CacheId_2 和 CacheId_3 的 M 状态读写请求从未触发
- 这表明在当前模型中存在缓存利用的不均衡

5.5 总体性能对比

根据分析结果，两个协议的总体性能指标对比如下：

协议	状态数	规则触发次数	验证时间
MSI	42,343	86,033	0.20s
MESI	42,342	86,032	0.24s
差异	-1	-1	+0.04s

表 6: MSI 与 MESI 协议性能对比

5.6 结论与启示

5.6.1 主要发现

1. **状态空间相似**：在简化模型中，MESI 协议的状态空间增长有限
2. **性能提升微小**：MESI 协议的理论优势在小规模模型中体现不明显
3. **复杂度增加**：MESI 协议确实增加了设计和验证的复杂度
4. **规则分布集中**：总线处理和 I 状态请求占据了大部分执行时间

5.6.2 实际应用启示

1. **规模效应**：MESI 协议的优势在大规模多核系统中更加明显
2. **工作负载相关**：协议选择应考虑具体的访问模式和共享特征
3. **验证重要性**：形式化验证对于复杂协议的正确性保证至关重要
4. **性能评估**：简单的状态数量不足以评估协议性能，需要考虑实际运行特征

6 实验总结

本次实验通过 Murphi 形式化验证工具成功实现并验证了 MSI 和 MESI 缓存一致性协议。

6.1 遇到的困难和解决

6.1.1 MESI 协议实现中的关键问题

在 MESI 协议实现过程中遇到了最主要的技术难题：

1. 不变式违反问题

问题描述：初次实现时，验证器报告"No simultaneous Modified/Exclusive and Shared states"不变式被违反，系统中同时存在 E/M 状态缓存和 S 状态缓存。

问题根源：在 BusRd 事务处理时，没有正确处理 E/S 状态的选择逻辑。原始代码只检查是否有缓存提供数据来决定新状态，忽略了 snoop 过程中其他缓存状态的变化。

解决方案：

- 添加了 CountSharedCache() 函数来实时统计共享缓存数量

- 修正 BusRd 事务处理逻辑，在所有 snoop 操作完成后再决定新状态
- 确保只有在没有其他共享缓存时才将新请求缓存转为 E 状态

2. 调试技巧学习
问题描述：面对复杂的状态转换错误，需要掌握有效的调试方法。
解决方案：

- 学会使用`./mesi -tv`命令查看详细执行轨迹
- 分析不变式违反的具体场景和状态转换序列
- 理解 Murphi 的执行模型和规则触发机制

6.2 实验收获

6.2.1 技术能力提升

1. **形式化验证技能：**掌握了 Murphi 工具的使用方法，包括模型编写、编译、验证和调试的完整流程。
2. **协议设计理解：**深入理解了缓存一致性协议的设计原理，特别是状态转换的复杂性和边界情况处理。
3. **系统级思维：**学会了从系统角度思考多核环境下的数据一致性问题，理解了性能与复杂度的权衡。
4. **调试分析能力：**通过分析协议验证失败的情况，培养了系统性的问题分析和解决能力。

6.2.2 理论与实践结合

1. **理论验证：**通过实际编写和验证协议代码，将课堂理论知识转化为可执行的模型，加深了对缓存一致性理论的理解。
2. **协议比较：**通过对比 MSI 和 MESI 协议的实现和性能，理解了协议演进的动机和优化思路。
3. **工程实践：**体验了形式化方法在计算机系统设计中的重要作用，理解了验证在复杂系统开发中的价值。

6.3 参考资料

1. Murphi 官方文档：Murphi3.1/doc/User.Manual
2. David E. Culler, Jaswinder Pal Singh. Parallel Computer Architecture: A Hardware/Software Approach
3. 课程提供的缓存一致性协议介绍资料